

EcoMove Backend

Full Chat History & Demo Video Script

CS2701 Level 5 Group Project — Group A2

Generated: 13 March 2026

Muhammad Amad Uddin
Brunel University London

Table of Contents

1. Database Check — PostgreSQL & PgAdmin Setup
2. Demo Video Script (First Version — Structured Outline)
3. Demo Video Script (Final Version — Full Speech Script)
 - 3a. Before You Press Record
 - 3b. Section 1 — Introduction
 - 3c. Section 2 — Spring Boot Application Structure
 - 3d. Section 3 — Physical Data Model in PgAdmin
 - 3e. Section 4 — Populated Database Data
 - 3f. Section 5 — Thunder Client API Demonstration
 - 3g. Section 6 — Key Technical Details
 - 3h. Section 7 — Closing
- 3i. Quick Reference — API Calls in Order

1. Database Check — PostgreSQL & PgAdmin Setup

■ USER

Check if I have postgres database and I have PGadmin 4 in the computer so show me some data in the pgadmin

■ CLAUDE (AI ASSISTANT)

PostgreSQL is running and your **ecomove** database has data. Here is a summary of what was found:

Database Details

Database: ecomove | Host: localhost | Port: 5432 | Username: postgres | Password: postgres

Tables Found

Table	Records	Description
users	4 users	Registered user accounts
transport_types	5 types	Car, Bus, Train, Bike, Walking
trips	5 trips	Logged journey records

How to View Data in PgAdmin 4

- Open PgAdmin 4 from Applications.
- Right-click **Servers** → **Register** → **Server**.
- Under the **General** tab, set Name: **EcoMove**.
- Under the **Connection** tab, set Host: **localhost**, Port: **5432**, Database: **ecomove**, Username: **postgres**, Password: **postgres**.
- Click **Save**.
- Navigate to: Servers → EcoMove → Databases → ecomove → Schemas → public → Tables.
- Right-click any table → **View/Edit Data** → **All Rows** to see the data.

■ Security note: test@example.com has a plain-text password (123456). This should be hashed if the project moves beyond development.

2. Demo Video Script — First Version (Structured Outline)

■ USER

So now I have to demonstrate a video explaining all the work done. Can you write a script to demonstrate just the back end if I upload the template to you.

■ USER

[University template uploaded — CS2701 Deliverable 4: Individual Demo Video. Requirements: show Spring Boot structure, physical data model, populated database, and API calls using Thunder Client. Duration ≤ 10 minutes. Face must be visible.]

■ CLAUDE (AI ASSISTANT)

Here is the full demo video script structured to the university requirements:

Thunder Client Requests — Quick Reference

#	Method	URL	Body
1	GET	http://localhost:8080/api/health/status	None
2	GET	http://localhost:8080/api/health/db	None
3	GET	http://localhost:8080/api/transport-types	None
4	POST	http://localhost:8080/api/trips	JSON body
5	GET	http://localhost:8080/api/users/1/stats	None
6	GET	http://localhost:8080/api/leaderboard	None
7	GET	http://localhost:8080/api/stats/summary	None

Timing Guide — First Version

Section	Duration
Introduction	0:30
Spring Boot Structure	1:30
PgAdmin — Data Model	1:30
PgAdmin — Populated Data	1:00
Thunder Client API Demo	3:00
Technical Highlights	1:00
Outro	0:30
Total	~9:00

3. Demo Video Script — Final Version (Full Speech Script)

■ USER

No — like a speech script with detailed instructions directed from one component to another.

■ USER

Make it a bit more clearer as this is a bit more messy. The text should be clearer and they should also show all the necessary steps.

■ CLAUDE (AI ASSISTANT)

Here is the full word-for-word speech script with clear stage directions at every step:

BEFORE YOU PRESS RECORD

Complete ALL of these steps before starting the screen recorder.

- Open VS Code with the **Brunel-Uni/backend** project.
- Run the backend — open the terminal in VS Code and run **mvn spring-boot:run**. Wait until you see *Started EcoMoveApplication* in the terminal.
- Open PgAdmin 4 and connect to the **ecomove** database.
- Open Thunder Client in VS Code (left sidebar icon).
- Make sure your face is visible in the corner of the recording.

SECTION 1 — INTRODUCTION

Duration: ~30 seconds | Camera on your face

■ *Look directly at the camera and say:*

"Hi, my name is Muhammad Amad Uddin. This is my individual backend demo for the EcoMove application.

EcoMove is a carbon footprint tracker. Users log their journeys — for example, a 10 kilometre bus trip — and the system automatically calculates how much CO2 they emitted and how much CO2 they saved compared to making that same journey by car.

In this demo I will cover four things:

- The Spring Boot application structure
- The database tables in PgAdmin
- The populated data
- And live API calls using Thunder Client

"Let me share my screen now."

■ *Start screen share — show VS Code*

SECTION 2 — SPRING BOOT APPLICATION STRUCTURE

Duration: ~2 minutes | Show VS Code Explorer

■ *In VS Code, click the Explorer icon on the left sidebar. Navigate to and expand: backend/src/main/java/com/ecomove/backend/*

"Here is the structure of my Spring Boot application. It follows the standard layered architecture with four main packages."

Step 1 — model/

■ *Click to expand the model/ folder. You should see: User.java, Trip.java, TransportType.java*

"This contains three JPA entity classes — User, Trip, and TransportType. Each class maps directly to a table in the PostgreSQL database. Hibernate reads these classes and automatically creates the table structure from them."

Step 2 — repository/

■ Click to expand the repository/ folder. You should see: `UserRepository.java`, `TripRepository.java`, `TransportTypeRepository.java`

"These are Spring Data JPA interfaces — one for each entity. They handle all database operations such as saving, finding, and deleting records. Inside `TripRepository` I have also written custom JPQL queries for more complex operations like calculating leaderboard rankings and aggregating statistics per user."

Step 3 — service/

■ Click to expand the service/ folder. You should see: `TripService.java`, `UserService.java`, `AuthService.java`, `StatsService.java`, `LeaderboardService.java`, `TransportTypeInitializer.java`

"This is where all the business logic lives. For example, `TripService` calculates the CO2 emitted and CO2 saved for every trip. `StatsService` aggregates platform-wide totals. And `TransportTypeInitializer` runs automatically on startup to seed the database with the five transport types."

Step 4 — controller/

■ Click to expand the controller/ folder. You should see: `TripController.java`, `UserController.java`, `AuthController.java`, `StatsController.java`, `LeaderboardController.java`, `TransportTypeController.java`, `HealthController.java`, `HealthApiController.java`

"These are the REST controllers that expose the API endpoints. Each controller maps incoming HTTP requests to the correct service method. I have controllers for trips, users, authentication, transport types, leaderboard, stats, and health checks."

Step 5 — dto/

■ Click to expand the dto/ folder. You should see: `LeaderboardDTO.java`, `UserStatsDTO.java`, `SummaryStatsDTO.java`

"DTO stands for Data Transfer Object. I use these to control exactly what gets returned in each API response, keeping the internal database models separate from what the client receives."

"So the full request flow is: a request hits a Controller, the Controller calls a Service, the Service uses a Repository to read or write the database, and the result travels back up and is returned as a JSON response."

SECTION 3 — PHYSICAL DATA MODEL IN PGADMIN

Duration: ~2 minutes | Switch to PgAdmin 4

■ Switch to PgAdmin 4

"Now let me show the physical database structure in PgAdmin. I am connected to the **ecomove** PostgreSQL database running on localhost port 5432."

Step 1 — Navigate to the tables

■ In the left panel expand in this order: Servers → *EcoMove* → Databases → *ecomove* → Schemas → *public* → Tables

"You can see there are three tables — `users`, `transport_types`, and `trips`. Let me go through each one."

Step 2 — Show the users table structure

■ Click on 'users' in the left panel, then expand 'Columns' underneath it

"The users table has four columns. **id** is the primary key, auto-incremented. **name** stores the user's display name. **email** has a unique constraint — no two accounts can share the same email address. And **password** stores the BCrypt-hashed password."

Step 3 — Show the transport_types table structure

■ Click on 'transport_types' in the left panel, then expand 'Columns' underneath it

"The transport_types table has three columns. **id** is the primary key. **name** is the transport mode — Car, Bus, Train, Bike, or Walking. And **co2_per_km** is the most important field in the application. It stores how many kilograms of CO2 that transport mode emits per kilometre. This single value drives every carbon calculation in the system."

Step 4 — Show the trips table structure

■ Click on 'trips' in the left panel, then expand 'Columns' underneath it

"The trips table has seven columns. **id** is the primary key. **user_id** is a foreign key linking to the users table. **transport_type_id** is a foreign key linking to the transport_types table. **distance_km** is how far the trip was. **co2_emitted** is the actual carbon emissions for that trip. **co2_saved** is how much CO2 was saved compared to making the same journey by car. And **trip_date** is automatically recorded when the trip is created."

SECTION 4 — POPULATED DATABASE DATA

Duration: ~1 minute | Stay in PgAdmin 4

Step 1 — Show transport_types data

■ Right-click on 'transport_types' → View/Edit Data → All Rows

"Here is the populated data in the transport_types table. Car emits 0.192 kg CO2 per km, Bus emits 0.105, Train emits 0.041, and Bike and Walking emit zero because they produce no carbon. These five rows are seeded automatically by the application every time it starts up."

Step 2 — Show users data

■ Right-click on 'users' → View/Edit Data → All Rows

"And here are the registered users in the system. You can see several test accounts with hashed passwords stored in the database."

Step 3 — Show trips data

■ Right-click on 'trips' → View/Edit Data → All Rows

"And here are the logged trips. Notice that **co2_emitted** and **co2_saved** are already calculated and stored for each row. For example, this 10 km bus trip shows 1.05 kg emitted and 0.87 kg saved — because choosing the bus instead of a car for 10 km avoids that much CO2."

Keep this trips table visible on screen. After the next API call you will see a brand new row appear here in real time.

SECTION 5 — THUNDER CLIENT API DEMONSTRATION

Duration: ~3 minutes | Switch to VS Code → Thunder Client

■ Switch to VS Code and click the Thunder Client icon in the left sidebar

"Now I will use Thunder Client to demonstrate the REST API. The backend is running on localhost port 8080. I will show seven endpoints."

API Call 1 — Health Check

■ Create a new request:

Method: GET

URL: `http://localhost:8080/api/health/status`

Click Send

"First, a basic health check. The API returns **ok: true**, confirming the backend is running and reachable."

API Call 2 — Database Health Check

■ Create a new request:

Method: GET

URL: `http://localhost:8080/api/health/db`

Click Send

"This endpoint verifies the database connection. It returns the database product name confirming we are connected to PostgreSQL."

API Call 3 — Get All Transport Types

■ Create a new request:

Method: GET

URL: `http://localhost:8080/api/transport-types`

Click Send

"This returns all five transport types from the database — their names and CO2 per km values. The frontend uses this response to populate the transport selection when a user logs a trip."

API Call 4 — Log a New Trip ★ Most Important

■ Create a new request:

Method: POST

URL: `http://localhost:8080/api/trips`

Click the Body tab → select JSON → paste this:

```
{
  "userId": 1,
  "transportTypeId": 3,
  "distanceKm": 15
}
```

Click Send — pause and look at the response

"This is the most important endpoint — logging a trip. I am sending a 15 kilometre train journey for user 1. Transport type 3 is the train. Let me send this now."

"Look at the response. **co2Emitted** is 0.615 — that is 15 km multiplied by the train's rate of 0.041 kg per km. **co2Saved** is 2.265 — that is because a car would have produced 2.88 kg for the same 15 km journey. By choosing the train, the user saved 2.265 kg of CO2. The trip date was recorded automatically. Status 200 OK."

■ Switch to PgAdmin — the trips table should still be open from Section 4

■ Right-click the trips table → Refresh, or click View/Edit Data → All Rows again

"And there is the new row at the bottom of the trips table — with exactly those values. This confirms the data was calculated in the service layer and correctly persisted to PostgreSQL."

■ Switch back to Thunder Client

API Call 5 — User Statistics

■ Create a new request:

Method: GET

URL: <http://localhost:8080/api/users/1/stats>

Click Send

"This endpoint returns aggregated statistics for user 1 — total trips, total distance, total CO2 emitted across all their trips, and total CO2 saved. This is powered by a custom JPQL aggregate query in TripRepository that sums all the user's trip data in a single database query."

API Call 6 — Leaderboard

■ Create a new request:

Method: GET

URL: <http://localhost:8080/api/leaderboard>

Click Send

"The leaderboard ranks every user by their total CO2 saved. Users who consistently choose eco-friendly transport appear higher. This is a custom query that groups trips by user, sums the CO2 saved column, and orders the results from highest to lowest."

API Call 7 — Summary Statistics

■ Create a new request:

Method: GET

URL: <http://localhost:8080/api/stats/summary>

Click Send

"Finally, the summary statistics endpoint. This returns platform-wide data — total trips logged by all users, total CO2 saved across the whole platform, number of active users, how many low-carbon trips were logged today, and the top three leaderboard entries. This is what a dashboard view would consume."

SECTION 6 — KEY TECHNICAL DETAILS

Duration: ~1 minute | Stay in VS Code

Step 1 — Open `TripService.java`

■ *Navigate to and open: `service/TripService.java`*

"The CO2 calculation logic is here in `TripService`. For every trip, the service multiplies the distance by the transport type's CO2 rate to get emissions. Then it calculates savings by comparing that figure against what a car would have emitted for the same distance. Car is always the baseline. If you walk or cycle, you save the full car emission amount."

Step 2 — Open `TripRepository.java`

■ *Navigate to and open: `repository/TripRepository.java`*

"In `TripRepository` I have written a custom JPQL query that uses a projection interface to return aggregate results directly as a DTO. This is more efficient than loading every individual trip record into memory just to calculate totals."

Step 3 — Open `TransportTypeInitializer.java`

■ *Navigate to and open: `service/TransportTypeInitializer.java`*

"This class implements `CommandLineRunner`, which means it executes once automatically when the application starts. It checks whether each transport type already exists in the database and either creates or updates it. This keeps the transport data consistent across all environments."

SECTION 7 — CLOSING

Duration: ~30 seconds | Face back on camera

■ *Look directly at the camera and say:*

"To summarise — I built the complete backend for EcoMove using Spring Boot 3 with a PostgreSQL database.

The architecture follows a clean layered structure: controllers handle HTTP requests, services handle business logic, and repositories handle all database access.

The core feature is the CO2 calculation engine that computes emissions and savings for every trip, aggregates user statistics, and powers a leaderboard to encourage greener travel choices.

All data is persisted in PostgreSQL and accessible through a REST API.

Thank you for watching."

QUICK REFERENCE — All API Calls In Order

#	Method	URL	Body
1	GET	<code>http://localhost:8080/api/health/status</code>	No body
2	GET	<code>http://localhost:8080/api/health/db</code>	No body
3	GET	<code>http://localhost:8080/api/transport-types</code>	No body

4	POST	http://localhost:8080/api/trips	JSON (see below)
5	GET	http://localhost:8080/api/users/1/stats	No body
6	GET	http://localhost:8080/api/leaderboard	No body
7	GET	http://localhost:8080/api/stats/summary	No body

JSON Body for API Call 4 (POST /api/trips)

```
{
  "userId": 1,
  "transportTypeId": 3,
  "distanceKm": 15
}
```

Timing Guide — Final Version

Section	Duration
Section 1 — Introduction	~0:30
Section 2 — Spring Boot Structure	~2:00
Section 3 — Physical Data Model in PgAdmin	~2:00
Section 4 — Populated Database Data	~1:00
Section 5 — Thunder Client API Demo	~3:00
Section 6 — Key Technical Details	~1:00
Section 7 — Closing	~0:30
Total	~10:00